

The Trillion Message Kafka Setup at Walmart



BYTEBYTEGO

OCT 01, 2024



212



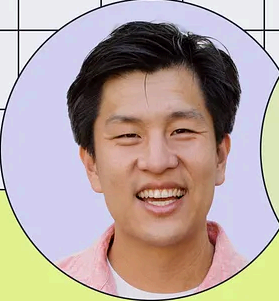
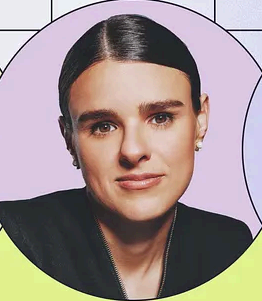
8



14

Share

[The Enterprise Ready Conference for engineering leaders \(Sponsored\).](#)

ENTERPRISE READY <i>Conf</i>	BY  WorkOS			
	WHERE <i>San Francisco</i>			
	WHEN <i>Oct 30</i>			
				
 Dropbox	 OpenAI	Vanta	checkr	<i>Canva</i>

The Enterprise Ready Conference is a one-day event in SF, bringing together product and engineering leaders shaping the future of enterprise SaaS.

The event features a curated list of speakers with direct experience building for the enterprise, including OpenAI, Vanta, Checkr, Dropbox, and Canva.

Topics include advanced identity management, compliance, encryption, and login — essential yet complex features that most enterprise customers require.

If you are a founder, exec, PM, or engineer tasked with the enterprise roadmap, this conference is for you. You'll get detailed insights from industry leaders that have years of experience navigating the same challenges you face today. And best of all, it's completely free since it's hosted by WorkOS.

Disclaimer: The details in this post have been derived from the Walmart Global Tech Blog. Credit for the technical details goes to the Walmart engineering team. The links to the original articles are present in the references section at the end of the post. We've attempted to analyze the details and provide our input about them. If you find any inaccuracies or omissions, please leave a comment, and we will do our best to fix them.

Walmart has a massive Apache Kafka deployment with 25K+ consumers across private and public cloud environments.

This deployment processes trillions of Kafka messages per day at 99.99% availability and supports critical use cases such as:

- Movement of data
- Event-driven microservices
- Streaming analytics

At Walmart's scale, the Kafka setup must be able to handle sudden traffic spikes. All consumer applications are written in multiple languages. Therefore, all consumer applications must adopt some best practices to maintain the same level of reliability and quality.

In this post, we'll look at the main challenges of a Kafka setup at this scale. Then, we will look at how Walmart's engineering team enhanced its setup to overcome these challenges and reliably process messages cost-efficiently.

Challenges with Kafka at Walmart's Scale

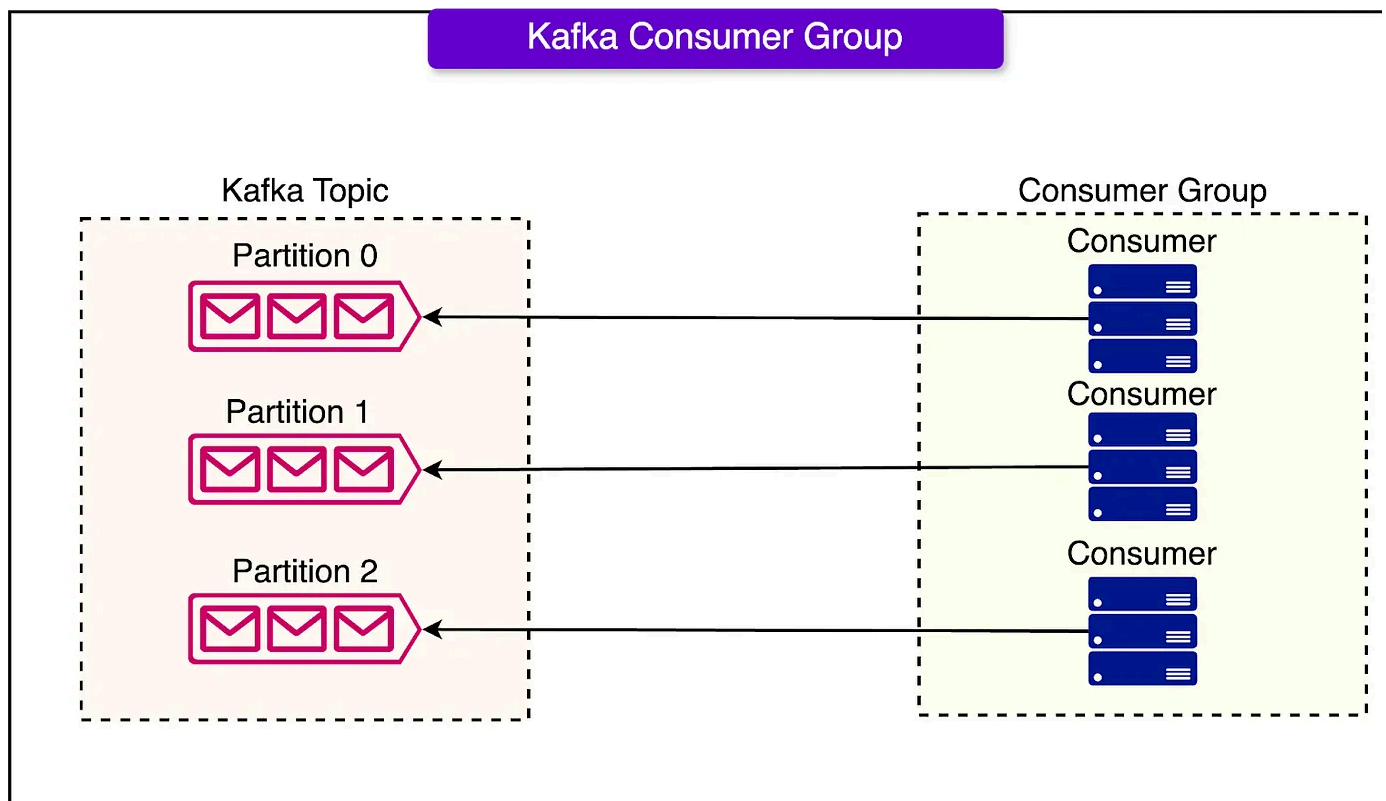
Let's start with understanding the main challenges that Walmart faced.

1 - Consumer Rebalancing

One of the most frequent problems was related to consumer rebalancing.

But what triggers consumer rebalancing in Kafka?

This can happen due to the changing number of consumer instances within a consumer group.



Several scenarios are possible such as:

- A consumer pod may enter or leave a consumer group. This can happen due to Kubernetes deployments, rolling restarts, or automatic scale-ins or scale-outs. Whenever it happens, Kafka needs to redistribute the partitions among the consumers.
- The Kafka broker may believe that a consumer has failed. If the broker has not received a heartbeat from a consumer within the configured session timeout, it assumes that the consumer has died. This can happen if the consumer's JVM errors or experiences a long stop-the-world garbage collection pause.
- The Kafka broker may believe that a consumer is stuck and trigger rebalancing if the consumer takes longer than a threshold to poll for the next batch of records; the broker marks it as stuck. This can happen when processing the previous batch takes too long.

Consumer rebalancing is needed to ensure partitions are evenly distributed. However, rebalancing can cause disruption and increased latency, particularly due to the near real-time nature of the e-commerce landscape.

2 - Poison Pill Messages

A “poison pill” message in Kafka is a message that consistently causes a consumer to fail when attempting to process it. This can happen due to various reasons such as:

- **Malformed Data:** The message payload may not be in an expected format. For example, invalid JSON or missing required fields. This may cause the consumer to throw an exception while processing.
- **Unexpected Data:** The message content might be syntactically valid but semantically incorrect. In other words, it might violate some business constraint.
- **Bugs in the Consumer Code:** If there's a bug (like a null pointer exception) in the code that handles the message, the processing will fail.

When the consumer encounters such a message, it will fail to process it and throw an exception. By default, the consumer will return to the broker to fetch the same batch of messages again. Since the poison pill message is still present in that batch, the consumer will again fail to process it, and this loop continues indefinitely.

As a result, the consumer gets stuck on this one bad message and is unable to make progress on other messages in the partition. This is similar to the “head-of-line blocking” problem in networking.

3 - Cost Concerns

There is a strong coupling between the number of partitions in a Kafka topic and the maximum number of consumers that can read from that topic in parallel. This coupling can lead to increased costs when trying to scale consumer applications to handle higher throughput.

For example, consider that you have a Kafka topic with 10 partitions and 10 consumer instances reading from this topic. Now, if the rate of incoming messages increases and the consumers are unable to keep up (i.e. consumer lag starts to increase), you might want to scale up your consumer application by adding more instances.

However, once you have 10 consumers (one for each partition) in a single group, adding more consumers to that group won't help because Kafka will not assign more than one consumer from the same group to a partition. The only way to allow more consumers in a group is to increase the number of partitions in the topic.

However, increasing the number of partitions comes with its challenges and costs.

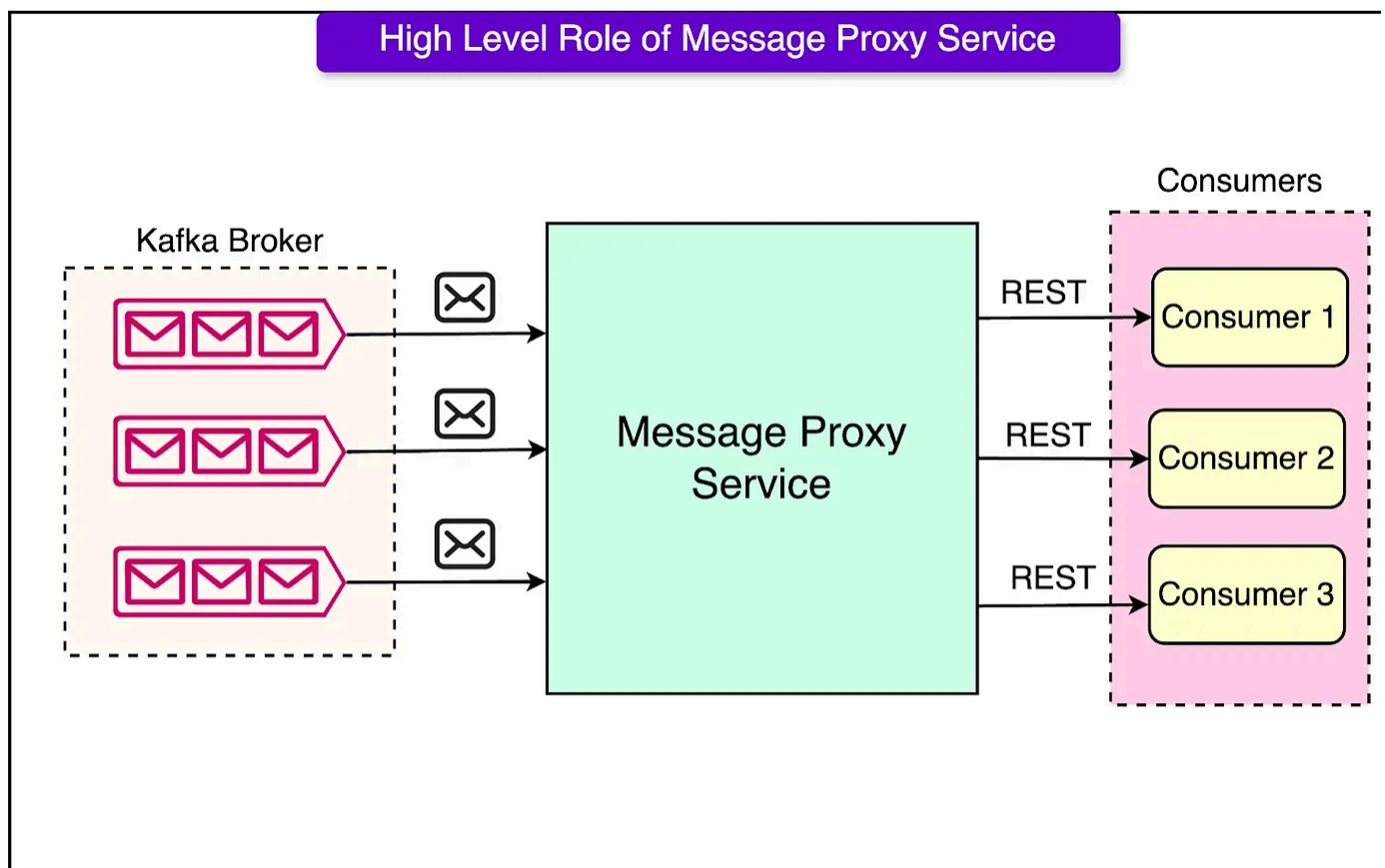
- Kafka has a recommended limit on the number of partitions per broker (for example, 4000 partitions per broker). If you keep increasing partitions, you may hit this limit and need to scale the Kafka brokers to larger instances, even if the brokers have sufficient resources to handle the current load. Scaling to larger broker instances is expensive.

- Increasing partitions requires coordination among the Kafka team, the producer and the consumer teams. In a large organization with thousands of Kafka pipelines, this coordination overhead is significant.
- More partitions also mean more open file handles, increased memory usage, and more threads on the Kafka brokers. This can lead to higher resource utilization and costs.

Designing the Messaging Proxy Service (MPS)

To overcome the challenges mentioned in the previous section, the Walmart engineering team designed a Message Proxy Service (MPS).

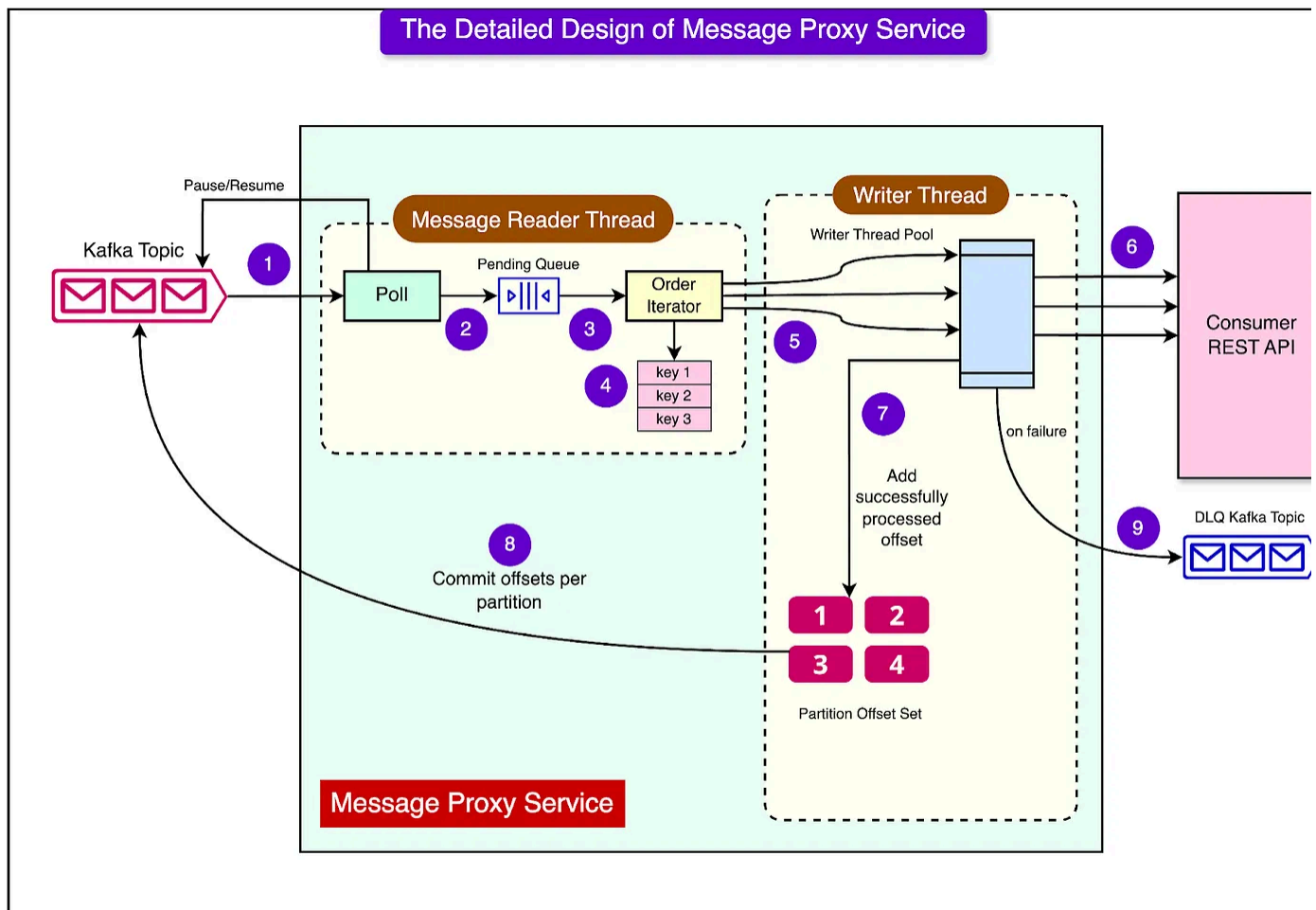
The diagram below shows a high-level view of MPS.



The MPS aims to decouple Kafka message consumption from the constraints imposed by Kafka's partition-based model. It works like this:

- MPS acts as a proxy between the Kafka brokers and the actual message consumer applications. It reads messages from Kafka partitions and puts them into a separate in-memory queue.
- Consumer applications don't directly read from Kafka. Instead, they receive messages from MPS via HTTP/REST. This allows the consumer applications to scale independently of the number of Kafka partitions.
- MPS ensures in-order message processing per key, handles consumer application failures, and manages offset commits back to Kafka.

The diagram below shows the detailed design of the MPS with all its components



Let's now look at the various components of the MPS in more detail.

Reader Thread

This is a single thread that reads messages from Kafka.

It writes the messages from the Kafka broker into a bounded queue called the "PendingQueue". If the PendingQueue reaches its maximum size, the reader thread will pause reading from Kafka.

This is a form of backpressure to prevent the queue from growing indefinitely if the writer threads cannot keep up.

Bounded Buffer Queue (PendingQueue)

This is a queue that sits between the reader thread and the writer threads. It has a maximum size to prevent it from consuming too much memory.

The PendingQueue allows the reader and write threads to work at different speeds. The reader can read messages as fast as Kafka can provide them, while the writers can process them at their own pace.

Order Iterator

This component ensures that messages with the same key are processed in the order they were received from Kafka.

It goes through the messages in the PendingQueue and skips any message if there is already an earlier message with the same key being processed. At any given time, at most one message per key is being handled by the writer threads.

Writer Threads

These are a pool of threads that take messages from the PendingQueue and send them to the consumer applications via HTTP POST requests.

If a POST request fails, the writer thread will retry the request a few times. If the retries are exhausted or if the consumer application returns certain HTTP codes, the writer thread will put the message into a Dead Letter Queue (DLQ).

The writer threads also help manage offsets. They update a shared data structure to keep track of processed offsets.

Offset Commit Thread

This is a separate thread that periodically wakes up (for example, every minute) and commits the Kafka offsets of processed messages using the Kafka consumer API.

It checks the shared data structure that is updated by the writer threads. Then, it commits the latest continuous offset for each partition. For example, if messages with offsets 1, 2, 3, and 5 have been processed for a partition, it will commit offset 3 (because 4 is missing).

By committing offsets periodically, MPS tells Kafka which messages it has processed successfully. If MPS crashes or is restarted, it will start consuming messages from the last committed offset, avoiding reprocessing messages that have already been handled.

Consumer Service REST API

This is the specification that the actual message consumer applications need to implement to receive messages from the MPS.

It defines the format of the HTTP POST request that the MPS writer threads will send (headers, body, etc.). It also specifies the meaning of different HTTP response codes that the consumer application can return.

See the table below that shows the API specification:

Type	Value	Description
Transport	HTTP/REST	
HTTP Accept	Accept=application/json	
	Accept-Charset=ISO-8859-1	
HTTP Headers	Key, Topic, Offset, Partition, All Kafka headers	
Body	payload: {Kafka message value encoded in ISO-8859-1}	
API	POST/process	
Return codes	2XX	Success
	4XX and 5XX	Proxy keeps retrying these return codes.
	6XX	Retry n' times (configurable) and then DLQ.
	7XX	DLQ. This indicates message format is BAD or failed business validation stage in the application.
	Connect & Read Timeouts	Auto DLQ. We see poison pill messages continuously timeout. MPS logic, based on the rate and duration of timeouts per message detects it to be a poison pill and automatically DLQs (Dead Letter Queue) it and alerts consumer application.

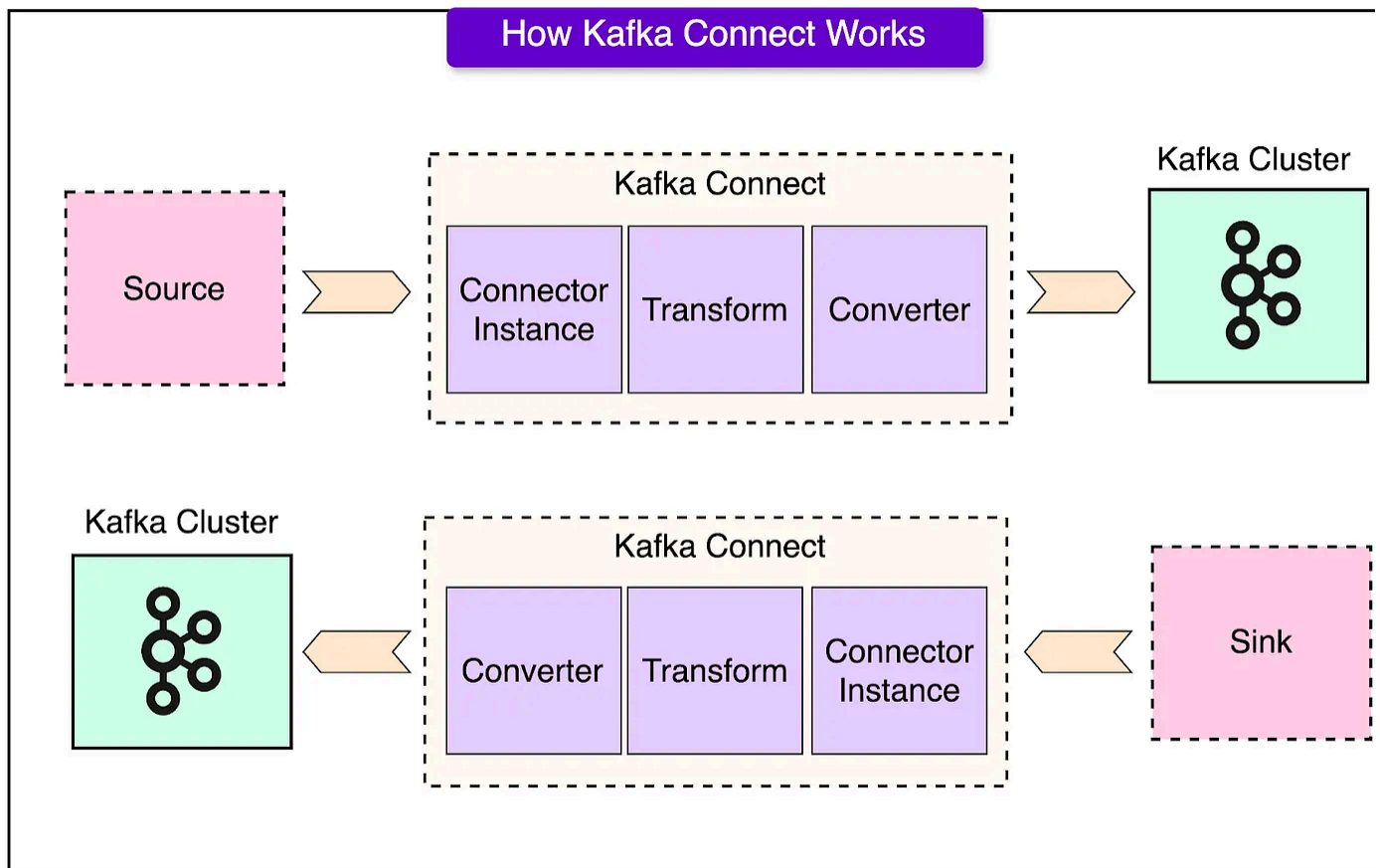
Source: [Walmart Tech Blog](#)

Implementation of MPS

MPS was implemented as a Kafka Connect sink connector.

For reference, Kafka Connect is a framework for connecting Kafka with external systems such as databases, key-value stores, search indexes, and file systems. It provides a standard way of defining connectors that move data into and out of Kafka.

The diagram below shows a high-level view of Kafka Connect



By implementing MPS as a Kafka Connect sink connector, the developers were able to use several features provided by the Kafka Connect Framework such as:

- **Multi-tenancy:** Kafka Connect allows running multiple connectors on a single cluster. This means that a single MPS deployment can serve multiple consumer applications (tenants).
- **Dead Letter Queue (DLQ) Handling:** Kafka Connect has built-in support for handling messages that cannot be processed.
- **Offset Commits:** Kafka Connect provides APIs for committing offsets. MPS uses these for its offset commit thread.
- **Scalability:** Kafka Connect is designed to be scalable and fault-tolerant. By building on top of Kafka Connect, MPS inherits these properties.

Apart from this, the consumer services (applications that process the messages) are designed to be stateless. This means they don't maintain any persistent state locally. Any state they need is either passed with the message or stored in an external database.

Being stateless allows these services to be easily scaled up or down in Kubernetes based on changes in message volume. If more messages need to be processed, Kubernetes can simply start more instances of the consumer service. If fewer messages are coming in, Kubernetes can terminate some instances to save resources.

Importantly, this scaling of the consumer services is independent of MPS and Kafka. MPS continues to read from Kafka and deliver messages to the consumer services regardless of how many instances of the consumer service are running.

Additional Points To Consider

Here are some additional points worth considering based on the MPS solution implemented by Walmart.

1 - Rebalancing of the MPS

MPS is also essentially a Kafka consumer. It reads messages from Kafka topics and makes them available to the application consumers via REST endpoints.

Like any other Kafka consumer, MPS would also be subject to rebalancing when the number of MPS instances changes. Based on the details, however, it seems that MPS is designed to handle rebalancing gracefully. The separation of the reader thread (which polls Kafka) and the writer threads (which send messages to the REST consumers) is the key here.

As long as MPS comes back up quickly after a rebalance, the REST consumers should be able to continue processing messages without substantial lag. The MPS design a

includes a bounded buffer (the PendingQueue) between the reader thread and write threads. This buffer helps to smoothen any temporary fluctuations in the rate at which MPS is reading from Kafka.

2 - Choice of REST

MPS calls REST APIs exposed by the consumer instances. Interestingly, the choice was REST and not something like gRPC.

This may be because of the simplicity of REST. Also, REST is widely supported by almost all languages and frameworks.

3 - Potential Increase in Complexity

While MPS solves several problems, it also introduces an additional layer to the system.

Instead of just having Kafka and the consumer applications, there is now a proxy service in the middle. This means more components to develop, deploy, monitor, and maintain.

Conclusion

The implementation of MPS helped Walmart achieve some key improvements.

- Most rebalances have now been eliminated except for rare restarts or network issues. With MPS, the reader thread consistently puts all polled messages into the PendingQueue within the allocated time. This prevents rebalances triggered by the Kafka broker thinking the consumer is stuck.
- Poison pill messages are handled in a better way. With MPS, consumer services can detect poison pill messages and notify MPS using specific HTTP return codes (600 and 700).

- MPS enables cost savings in multiple ways. Firstly, consumer services are now stateless and can scale quickly in Kubernetes based on demand. They don't need to be scaled in advance. Secondly, Kafka clusters can be scaled based on throughput rather than the number of partitions.

References:

- [Reliably Processing Trillions of Kafka Messages Per Day](#)
- [Kafka Connect](#)

SPONSOR US

Get your product in front of more than 1,000,000 tech professionals.

Our newsletter puts your products and services directly in front of an audience that matters - hundreds of thousands of engineering leaders and senior engineers - who have influence over significant tech decisions and big purchases.

Space Fills Up Fast - Reserve Today

Ad spots typically sell out about 4 weeks in advance. To ensure your ad reaches this influential audience, reserve your space now by emailing

sponsorship@bytebytego.com



212 Likes • 14 Restacks

Discussion about this post

Comments Restacks



Write a comment...



cch  Dec 26, 2024

Not sure if they have considered this library before which helps processing messages in parallel. Having a MPS can possibly be avoided. <https://github.com/confluentinc/parallel-consumer>

 LIKE  REPLY



Hamza Nov 28, 2024

MPS should be a Kafka connect source connector, right ? Or I didn't understand correctly

 LIKE  REPLY



6 more comments...

© 2026 ByteByteGo · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great culture